

A Systematic Review of Modern Object Detection: Classical CNN Architectures and Transformer-Based Approaches

Amer Sh. Mohammed

Faculty of Radio Engineering and Telecommunications
Saint Petersburg Electrotechnical State University (ETU "LETI")

St. Petersburg, Russia

amokhammed2@stud.etu.ru

Abstract—Object detection, a fundamental task in computer vision, has undergone a revolutionary transformation with the advent of deep learning. This paper provides a comprehensive review of the key architectural paradigms that have shaped the field. We begin by examining the two-stage detection framework, initiated by the Region-based Convolutional Neural Network (R-CNN) and evolving through Fast R-CNN to the end-to-end Faster R-CNN with its Region Proposal Network (RPN). The review then analyzes the one-stage detection framework, which prioritizes speed for real-time applications. This includes the You Only Look Once (YOLO) family, which formulates detection as a single regression problem, and the Single Shot MultiBox Detector (SSD), which employs predefined anchor boxes across multi-scale feature maps. Furthermore, we explore the recent paradigm shift driven by attention mechanisms, detailing the evolution from foundational self-attention modules integrated into CNN backbones to the Vision Transformer (ViT) and subsequent detection-specific architectures like DETR and Deformable DETR, which leverage transformer encoders and decoders for set-based prediction. For each paradigm and model, we discuss core conceptual breakthroughs, key architectural components, and inherent trade-offs between accuracy, speed, and computational complexity. Finally, we synthesize the comparative strengths and limitations of these landmark approaches and outline emerging trends and future directions in deep learning-based object detection. This survey serves as a structured guide to the foundational and transformative models that define the state-of-the-art in this rapidly advancing domain.

Keywords—Object Detection, Deep Learning, Computer Vision, R-CNN, Faster R-CNN, YOLO, Single Shot MultiBox Detector (SSD), Two-Stage Detection, One-Stage Detection, Architectural Evolution, vision transformer, self-attention mechanism.

I. INTRODUCTION

One of the key components of contemporary artificial intelligence is object detection, which is the computer vision task of locating and identifying things within an image [1]. Robust object detection is essential to robots' ability to "see" and understand their surroundings, from the safety features in self-driving cars [2], and the accuracy of medical imaging [3] to the ease of automated retail and reliable surveillance systems [4]. But how did we get to today's extremely precise, real-time systems? This study traces the development from conceptual breakthroughs to useful, cohesive systems, reviewing the groundbreaking deep learning architectures that sparked a dramatic shift in this discipline. Object detection was mostly dependent on manually created features and complex machine learning pipelines before deep learning took over. Methods such as the Histogram of Oriented

Gradients (HOG), when combined with classifiers like Support Vector Machines (SVMs) [5], or the Viola-Jones framework for face detection [6], were widely used. Despite these techniques being novel at the time, they were frequently brittle, required a high level of domain expertise to build, and had trouble with the wide range of object heterogeneity in real-world settings. Their performance reached a level, which made the conditions for a paradigm change favourable. The trade-off between processing speed and accuracy has shaped the development of deep learning-based object detection, leading to the development of two distinct paradigms:

The Two-Stage Paradigm: High accuracy is the top priority for this class of detectors. It works in two steps:

(1) Creates a number of region proposals that may contain objects.

(2) Classifies each proposal and fine-tunes its bounding box. Although this method is very accurate, it is computationally demanding [7].

The One-Stage Paradigm: This framework enables real-time detection by emphasizing efficiency and speed [8]. By reformulating detection as a single regression or classification problem over the image, it eliminates the region proposal phase. Faster inference results from this, although historically, accuracy has been sacrificed in comparison to two-stage approaches. The two-stage story is one of progressive refinement and integration. It began with R-CNN (Region-based CNN), which, despite its high cost, proved the profound potential of CNNs for detection. Its successor, Fast R-CNN, introduced a more efficient architecture by sharing convolutional computations across region proposals. This evolution culminated in Faster R-CNN, which integrated the region proposal mechanism directly into the network via a Region Proposal Network (RPN), enabling true end-to-end training and solidifying the two-stage approach as the benchmark for accuracy. The need for real-time performance in applications like robotics and video analysis gave rise to the one-stage paradigm. By defining detection as a single, cohesive regression task from image pixels to spatial separations and class probabilities, the YOLO (You Only Look Once) family of models proved revolutionary. Simultaneously, the SSD (Single Shot MultiBox Detector) architecture achieved a balance by combining multi-scale feature maps and pre-defined anchor boxes, providing an attractive substitute that combined YOLO speed with competitive accuracy. Understanding each model's legacy and significance requires a rigorous examination of the speed-versus-accuracy trade-off. In addition, the field has continued to grow quickly. The subsequent paradigm shift arrived with the Vision

Transformer (ViT). By segmenting images into patches and processing them with self-attention, ViT demonstrated that pure transformer architectures could achieve state-of-the-art performance on image classification without convolutional inductive biases. This success directly challenged CNNs and catalyzed the development of end-to-end transformer-based object detection models. In addition to summarizing the relative advantages and disadvantages of these seminal architectures, this review will briefly examine the contemporary trends they sparked, such as transformer-based models and anchor-free techniques, that indicate to the direction of object detection. The remaining part of this article is structured as follows. In Section 2. The background information on basic ideas and measures is given the development of two-stage detectors from R-CNN to Faster R-CNN is described in depth in Section 3. The emergence of one-stage detectors, specifically SSD and YOLO, Self-Attention and Transformers is examined in Section 4. The inherent trade-offs are discussed and compared in Section 5. wraps up the survey, while Section 6 describes present trends and potential directions.

II. BACKGROUND AND FUNDAMENTALS

Establishing a consistent technical terminology is crucial to comprehending the mechanics and evolution of the object detection models covered in this survey. The basic ideas, measurements, and architectural elements that support contemporary deep learning-based object detectors are described in this section.

A. The object detection task's definition
Object detection involves recognizing objects within an image using a bounding box and categorizing them with a class name [9].

B. Evaluation of the bounding box's accuracy.

The accuracy of a predicted bounding box in comparison to the ground truth is measured by the Intersection over Union (IoU) metric. It is the region of overlap divided by the area of union. $\text{IoU} \geq 0.5$ is a common threshold for an accurate detection. $\text{IoU} = \text{Area of Overlap} / \text{Area of Union}$ [10].

C. Standard metrics for evaluating an object detector's overall performance

Mean Average Precision (mAP) is the main metric. Precision and Recall: These core ideas form the foundation of mAP. The area under the Precision-Recall curve that summarizes the detection quality for a particular class is referred to as Average Precision (AP). mAP: The mean of AP for every type of object. mAP is averaged over several

IoU thresholds for more stringent examination (e.g., MS COCO) [11].

D. The speed of an object detector quantified

Frames-Per-Second (FPS) is a unit of measurement for inference speed that shows how many images a model can process in a second. This frequently entails a trade-off with mAP and is essential for real-time applications.

E. The key architectural components common to many detectors

- **Backbone Network** A CNN that has been pre-trained to extract hierarchical feature maps from the input image, such as ResNet..
- **Neck:** a component (such as a Feature Pyramid Network) that improves backbone features, frequently by combining various resolutions to improve multi-scale detection.
- **Head:** The last component of the network that carries out regression (improving bounding box coordinates) and classification (predicting class scores).the importance of "anchor boxes"

F. To fit objects and predict their class, the network learns to make small modifications (offsets) to these anchors, which are a collection of preconfigured boxes with different sizes and aspect ratios. This simplifies the learning process [12].

G. The necessary of Non-Maximum Suppression (NMS)

Duplicate detections for the same object are eliminated in a post-processing step. All bounding boxes with a high IoU overlap are suppressed while the highest-confidence bounding box is retained.

With these fundamentals established, we can explore the architectural evolution of object detectors [13].

III. THE TWO-STAGE DETECTOR EVOLUTION

A two-stage approach that put state-of-the-art accuracy ahead of computational performance characterized the first generation of deep learning-based object detectors. This paradigm is based on a straightforward however effective principle: first, identify regions of interest that may include objects, then carefully categorize and refine those regions. This paradigm's development—from R-CNN to Faster R-CNN—is the history of progressively combining various parts into a single, fully trainable system.

1. R-CNN:

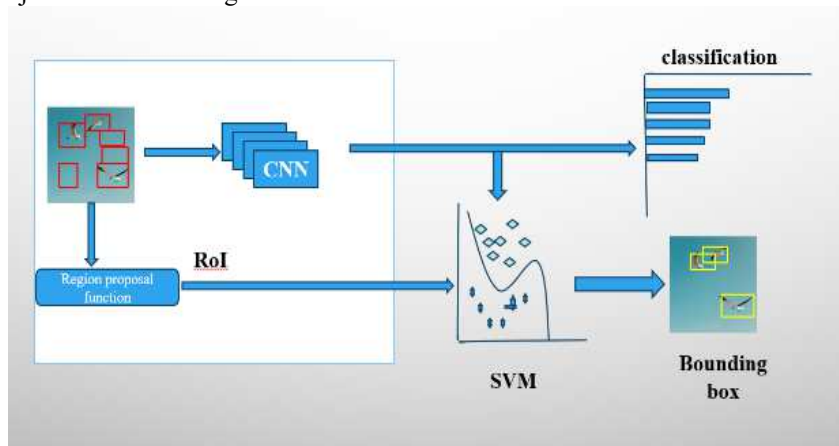


Fig. 1. RCNN architecture

The Innovative Proof of Concept When Girshick et al. presented the Region-Based Convolutional Neural Network (R-CNN) in 2014, it represented a significant change from earlier techniques and showed how CNNs may revolutionize object detection.

The basic idea behind R-CNN was to replace the manually created features employed in earlier techniques with a high-capacity CNN to extract features from image areas. Instead of using a CNN to look at the whole image at once for detection, it used a strong feature extractor for candidate regions. The R-CNN procedure was a multi-phase, sophisticated pipeline:

Region Proposal: From an input image, about 2000 category-agnostic region suggestions were produced using an external method known as Selective Search. This method did not require learning and was based on conventional computer vision.

Feature Extraction: To extract a fixed-length feature vector, each suggested region was warped (or cropped and scaled) to a given size (such as 227x227 pixels) and fed forward into a CNN that has already been trained, such as AlexNet. **Classification:** To identify the object class within each region, a set of class-specific Support Vector Machines (SVMs) were fed the collected characteristics [14].

Bounding Box Refinement: To increase localization accuracy, the bounding box coordinates for each class were refined using a different linear regression model

- R-CNN's main advantages and disadvantages.

Key Contributions: R-CNN demonstrated the superiority of deep features for detection by achieving a significant improvement in mAP on benchmarks such as PASCAL VOC (e.g., from 35.1% to 53.7%).

Serious Restrictions:

Extremely Slow and Computationally Expensive: Because overlapping regions would need to be processed more than once, extracting features from each of the approximately 2000 region suggestions separately resulted in enormous duplication of effort. There were several stages in the training process.

Memory-hungry: Every region proposal's features have to be kept on disk [15].

Not Completely Trainable: Because the CNN, SVMs, and regressors were trained separately, the system as a whole was not as good as it may have been.

2. Fast R-CNN:

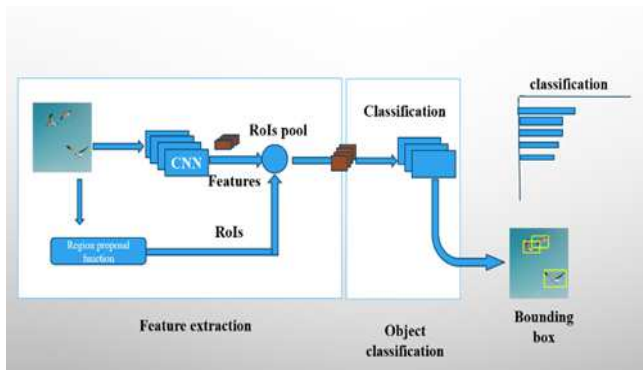


Fig. 2. Fast RCNN architecture

By using a backbone to process the entire image a , Fast R-CNN addressed R-CNN's computational bottleneck. Instead of processing each area proposal independently, CNN will provide a shared feature map. Regions of Interest (RoIs) were created by projecting region ideas from an external algorithm (such as Selective Search) onto this map. Each variable-sized RoI was then transformed into a fixed-size feature vector by the innovative RoI Pooling layer.

A single network with two parallel output heads received these features fully-connected layers these features:

1. A classification head that output a discrete probability distribution.

$$\mathcal{P} = \mathcal{P}_0, \mathcal{P}_1, \dots, \mathcal{P}_K \quad (1)$$

Over k objects class and background class.

2. A bounding-box regression heads that output class-specific refinements, $\mathbf{t}^k = (t_x^k, t_y^k, t_w^k, t_h^k)$

To adjust a position and size of the proposal.

3. Loss function in fast RCNN:

$$\mathcal{L}(\{p_i\}, \{t_i\}) = \frac{1}{N_{cls}} \sum_i \mathcal{L}_{cls}(p_i, t_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* \mathcal{L}_{reg}(t_i, t_i^*) \quad (3)$$

4. Bounding box regression

$$\begin{aligned} t_x &= (x - x_a) / w_a, t_y = (y - y_a) / h_a \\ t_w &= \log\left(\frac{w}{w_a}\right), t_h = \log\left(\frac{h}{h_a}\right) \\ t_x^* &= \left(x_0 - \frac{x_a}{w_a}\right), t_y^* = \left(y - \frac{y_a}{w_a}\right) \\ t_w^* &= \log\left(\frac{w^*}{w_a}\right) \end{aligned} \quad (4)$$

The symbols represent scale-invariant transformation parameters for bounding box regression. The predicted offsets (t_x, t_y, t_w, t_h) adjust a region proposal (x_a, y_a, w_a, h_a) toward the final detection, while the target offsets $(t_x^*, t_y^*, t_w^*, t_h^*)$, λ balancing hyperparameter [16]. define the ideal transformation to match the ground-truth box

3. Faster R-CNN:

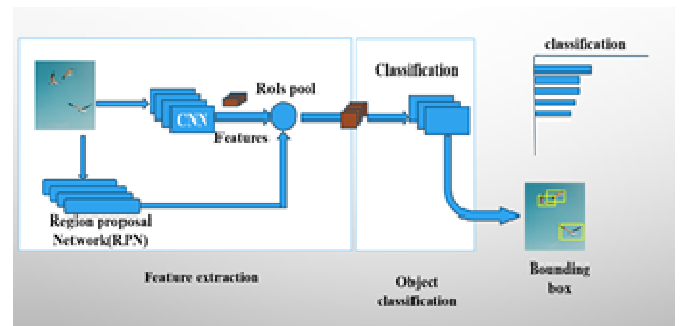


Fig. 3. Faster R-CNN architecture

Despite Fast R-CNN represented a significant advancement, the new computational bottleneck was based on an external, slow region proposal approach like Selective Search. Internalizing the region proposal phase alone was an important achievement for Faster R-CNN. The Region Proposal Network (RPN), a fully convolutional network that learnt to produce excellent region proposals straight from the feature map, was introduced by Faster R-CNN. This was the last stage in developing a deep learning-based object detector that was fully end-to-end.

Anchors: Anchor boxes are a concept used by the RPN. It takes into account k (e.g., 9) distinct anchor boxes with varied sizes and aspect ratios at each sliding window point on the feature map. As a result, a dense set of potential ideas encompassing the image at different scales and places is produced. Each Anchor Has Two Outputs: The RPN concurrently outputs the following for every anchor:

a "objectness" score (the likelihood that an object is present in the anchor as opposed to background). The proposal's refined bounding box coordinates (offsets with respect to the anchor).

Efficiency: The RPN generates suggestions at a low cost since it shares the full-image convolutional features with the detection network (Fast R-CNN). The Faster R-CNN architecture as a whole is a single network.

A backbone processes the image. CNN to create a feature map.

A set of object proposals with an "objectness" score is produced when the RPN moves across this feature map. For every proposal, fixed-size feature vectors are extracted by the RoI Pooling layer. The Fast R-CNN head receives these characteristics for bounding-box regression and final classification. Alternating Training, in which the RPN and Fast R-CNN components are trained in alternating stages while sharing and updating the convolutional features, is a crucial aspect of the training procedure. Almost Real-Time Speed: The speed increased to 5–17 FPS on a GPU since the region proposal step is now nearly free.

Cutting Edge Accuracy: It became the unquestioned best two-stage approach and a standard baseline for years to come, setting a new norm for accuracy on public datasets.

End-to-End Paradigm: It established a strong, long-lasting architectural pattern by completing the transition from a disjointed pipeline to a cohesive, deep learning model [16].

IV. THE RISE OF ONE-STAGE DETECTORS

The multi-stage conventional approach was challenged by a new family of detectors that developed as a result of real-time high-performance applications like robotics and video analysis. In order to achieve localization and classification in a single, continuous pass over the image, the one-stage paradigm does away with the separate region proposal step. This change puts rapidity and simplicity of use first, frequently at the expense of accuracy, which later versions have diligently tried to reduce.

1. The Unified Detection Paradigm, or YOLOv1:

By developing a unified, single neural network that can predict bounding boxes and class probabilities from all of an image in a single evaluation, OLO transformed object detection. Compared to earlier techniques, this process was much quicker and easier.

B bounding boxes, each having an IoU and objectness probability represented by a confidence score. a collection of class probabilities that depend on the presence of an object. A multi-part loss function is used to train the model in its entirety, combining:

Mean Squared Error (MSE) on the bounding box coordinates is the localization loss. Confidence Loss: MSE for the objectness score, applied to both object-containing and object-free boxes (the latter having a lower weight). MSE for the class probability is the classification loss.

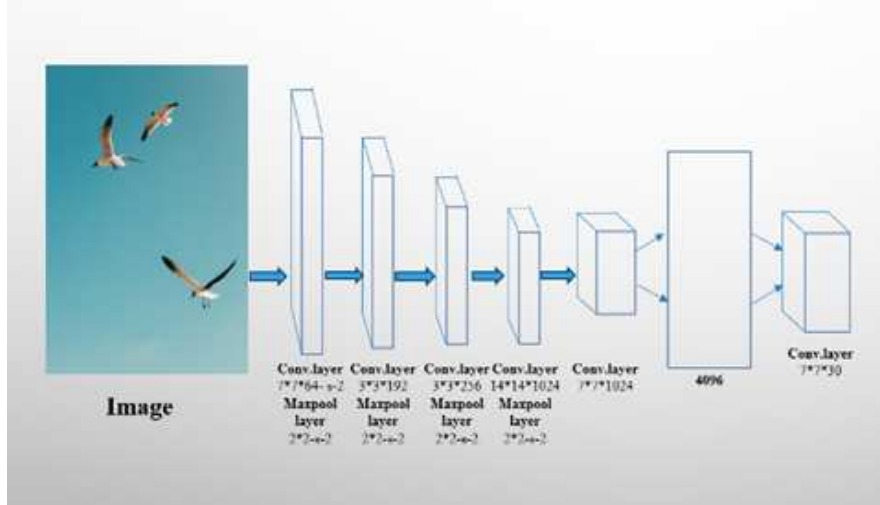


Fig. 4. The YOLOv1 architecture.

The whole YOLOv1 loss function is as follows:

$$L_{YOLO} = \lambda_{coord} \sum_{i=0}^S \sum_{j=0}^B 1_{ij}^{obj} ((x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2) + \sum_{i=0}^S \sum_{j=0}^B 1_{ij}^{obj} (c_i - \hat{c}_i)^2 + \lambda_{coord} \sum_{i=0}^S \sum_{j=0}^B 1_{ij}^{noobj} (c_i - \hat{c}_i)^2 + \lambda_{coord} \sum_{i=0}^S \sum_{j=0}^B 1_{ij}^{noobj} (p_i(c) - \hat{p}_i(c))^2$$

($\hat{x}_i, \hat{y}_i, \hat{w}_i, \hat{h}_i$): The predicted bounding box's center coordinates (relative to the grid cell) and its width/height (relative to the whole image).

($\hat{x}_i, \hat{y}_i, \hat{w}_i, \hat{h}_i$): The target/ground truth values for the box. **S**: The grid size. The image is divided into an $S \times S$ grid (e.g., 7×7 in YOLOv1). **B**: The number of Bounding box predictions each grid cell. **i**: Indexes over the S^2 grid cells. **j**: Indexes over the B bounding boxes predicted by a single grid cell. **c**: Indexes over the object classes. 1_{ij}^{obj} : This is 1 if any ground-truth object's center falls into grid cell i^* . Otherwise, it is 0. It activates the classification loss for that cell. 1_{ij}^{noobj} : This is 1 for bounding box predictors that are not responsible for an object [17].

• YOLOv8:

A significant improvement of the YOLO architecture is YOLOv8 was created by Ultralytics and established a new benchmark for effectiveness and usability in real-time object detection. Its refined, solely convolutional architecture, which is based on a CSPDarknet backbone improved with C2f (Cross Stage Partial fused) modules, its main innovation. By adding more skip connections and split operations, these modules take the place of the previous C3 modules, enabling richer gradient flow and more efficient feature fusion. Its switch to an anchor-free detection head, which predicts an item's center directly rather than using predefined anchor boxes, is a significant improvement that

simplifies the model and enhances its capacity to generalize across a range of object sizes. Additionally, YOLOv8 uses a decoupled head, which divides the bounding box regression and classification responsibilities, leading to more reliable and efficient training. When combined with sophisticated loss functions like Distribution Focal Loss (DFL) and CIOU for localization, as well as a dynamic Task-Aligned Assigner for label matching, YOLOv8 produces a potent, user-friendly package that strikes a balance between high speed and cutting-edge accuracy, solidifying its status as a flexible workhorse for real-world applications. [18].

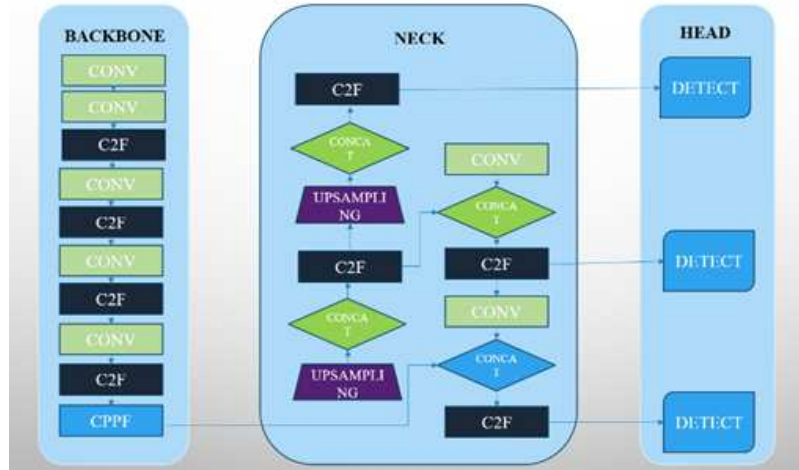


Fig. 5. The YOLOv8 architecture.

• YOLOv11:

In order to push the limits of contextual comprehension and accuracy, Ultralytics' following evolution represents a purposeful shift from a solely convolutional network to a hybrid CNN-Transformer design that incorporates attention processes. The deep integration of self-attention and transformer-inspired modules, such as the innovative C3k2 or comparable hybrid blocks in its neck and backbone, into its fundamental architecture is its most distinctive added feature. These blocks help the model better resolve complex images with occlusions or small objects by combining convolutional layers for local feature extraction with

effective attention mechanisms ("k2") that model long-range dependencies throughout the image. Next-generation approaches to training, such as higher-level dynamic label assignment and sophisticated loss functions tuned for the hybrid output, facilitate this architectural change. By giving the network a global receptive field, YOLOv11 clearly addresses the fundamental restriction of CNNs and achieves higher performance on demanding benchmarks while maintaining the user-centric philosophy and decoupled head structure of YOLOv8. As a result, YOLOv11 serves as evidence of the revolutionary influence of attention mechanisms, pointing the YOLO lineage in the direction of a future in which contextual thinking is crucial. [19].

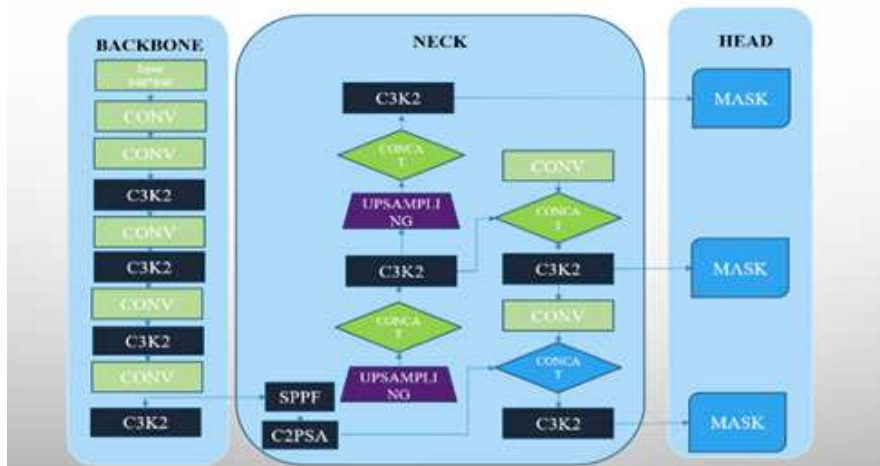


Fig. 6. The YOLOv11 architecture

2. Self-Attention and Transformers: Enabling Global Context

While self-attention and the Transformer architecture originated in natural language processing, they have

profoundly impacted computer vision. These mechanisms enable models to capture global context and long-range dependencies across an image—a capability traditional CNNs lack due to their inherently local receptive fields. Through self-attention, each element of an image, such as a

patch, can dynamically assess and incorporate information from every other patch. This process works by projecting input features into Query (Q), Key (K), and Value (V) vectors. Attention weights, derived from the similarity between Queries and Keys, are used to compute a weighted sum of Values. This allows the model to focus on semantically relevant connections, such as linking a dog's ears and tail across the scene, while suppressing irrelevant background clutter, leading to more robust and context-aware feature representations. The Transformer architecture implements this through Multi-Head Self-Attention, which runs multiple attention processes in parallel to capture diverse types of interactions, and uses positional encodings to preserve spatial information that would otherwise be lost in the set-based processing of image patches. Pure Vision Transformers (ViTs) leverage this architecture to treat an image as a sequence of patches, achieving strong performance on large-scale datasets, though they can be computationally demanding and data-hungry compared to convolutional counterparts [28].

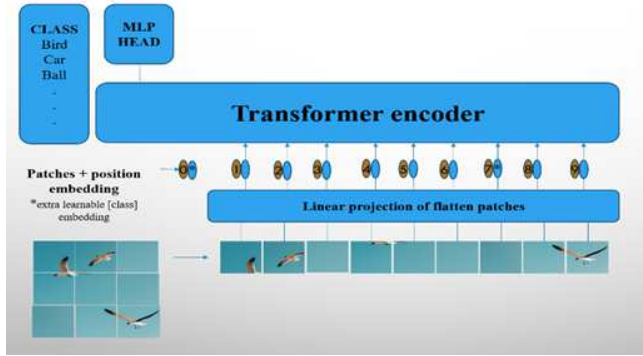


Fig. 7. Vision transformer architecture

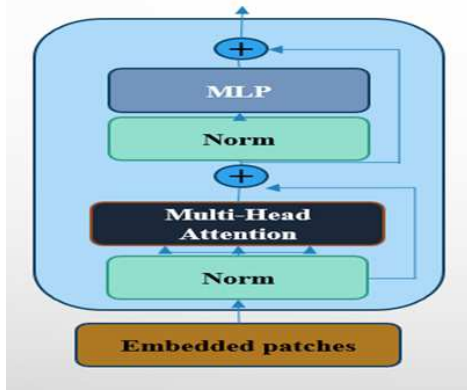


Fig. 8. Transformer encoder

3. SSD: A Pragmatic Balance of Concepts

The Single Shot MultiBox Detector (SSD) emerged as a compelling one-stage architecture that combined concepts from YOLO's speed and Faster R-CNN's accuracy. Making multi-scale predictions from several feature maps was its primary innovation, which enabled it to efficiently identify items of different sizes—small things on higher-resolution maps and large objects on lower-resolution ones. SSD tiled these maps with default boxes (anchors) of various sizes and aspect ratios. In order to fine-tune the location of each default box, the network predicted both class scores and offsets. Using a multi-task loss that combined localization and classification, the model was trained end-to-end:

$$L(x, l, g) = \frac{1}{N} (L_{conf}(x, c) + \alpha L_{loc}(x, l, g)) \quad (6)$$

$$L_{loc}(x, l, g) = \sum_{i \in pos} \sum_{m \in \{cx, cy, w, h\}} x_{ij}^k \left(smooth_{l1}(l_i^m - g_j^m) \right) \quad (7)$$

$$L_{conf}(x, c) = - \sum_{i \in pos} x_{ij}^p \log(\hat{c}_i^p) - \sum_{i \in neg} \log(\hat{c}_i^0) \quad (8)$$

L : Total multi-task loss. N : Number of matched default boxes (normalization term). α : Weight balancing classification vs. localization loss. L_{conf} : Classification (confidence) loss component. L_{loc} : Bounding-box regression (localization) loss component. $i \in pos$: Summation over positive default boxes (matched to a ground truth object). $m \in \{cx, cy, w, h\}$: The four bounding box parameters (center-x, center-y, width, height). x_{ij}^k : Matching indicator (1 if default box i is matched to ground truth j of class k). l_i^m : Predicted offset for coordinate m . g_j^m : Target offset (encoded ground truth) for coordinate m . $smooth_{l1}(l_i^m)$: Robust regression loss function (less sensitive to outliers than MSE). $i \in pos / i \in neg$: Summation over positive (object) and negative (background) default boxes. $\hat{c}_i^p$: Predicted probability that box i contains an object of class p . $\hat{c}_i^0$: Predicted probability that box i is background (class 0). $\log$: Cross-entropy loss function, penalizing incorrect class predictions. [21]

V. COMPARATIVE ANALYSIS AND DISCUSSION

After studying each of the core architectures independently, this section outlines their relative advantages and drawbacks. In basic terms, the development of object detection is an explanation that involves technological trade-offs, primarily between accuracy and speed. The analysis that follows summarizes these trade-offs and highlights the conceptual developments that defined each era. Object detection model optimization is fundamentally limited by a performance trade-off: increasing computing efficiency (high FPS) usually requires reducing localization and classification accuracy (low mAP), and vice versa. Two-stage detectors (Faster R-CNN) consistently occupied the lower-speed, high-accuracy end of the spectrum. Their "coarse-to-fine" approach, which included a distinct region refining stage, enabled superior bounding box precision and management of difficult situations. However, one-stage detectors (YOLO, SSD) dominated the high-speed end, sacrificing some accuracy to analyse video in real-time and creating a new class of applications. The Evolution and Simplification of Architecture: The historical literature shows a clear shift toward more coherent and end-to-end trainable models. Beginning with the complex, multi-stage R-CNN pipeline (CNN + SVMs + Regressors), the process developed into the fully integrated Faster R-CNN. In the same manner, the one-stage family began with the delightfully simple YOLOv1 and through generations of improvement. This trajectory is demonstrated by more recent versions like YOLOv8 and YOLOv11, which combine complex components like decoupled heads, hybrid attention mechanisms, and anchor-free detection to achieve exceptional performance while preserving a cohesive, single-stage process that prevents a return to multi-stage complexity. Current Trends and Future Directions

VI. CURRENT TRENDS IN OBJECT DETECTIONS

With an emphasis on efficiency and novel structures, research is still moving beyond basic models. Anchor-Free Detectors: By predicting object centers or keypoints, models such as FCOS [22] and CenterNet eliminate the need for manually modified anchor boxes, streamlining the pipeline and reducing computing overhead. Transformer Revolution: By approaching detection as a direct set prediction problem, architectures like DETR eliminate the requirement for anchors and Non-Maximum Suppression (NMS) [23]. Early problems with convergence and small objects have been resolved by later advancements (such as Deformable DETR) [24]. YOLO's evolution: For real-time applications, the YOLO lineage continues to be dominant. Practical deployment frameworks and the methodical accumulation of numerous small, effective improvements—rather than a single architectural breakthrough—are what propel progress. Lightweight and Efficient Models: For edge deployment, research focuses on Neural Architecture Search (NAS) and model compression techniques (pruning, quantization) to optimize performance within strict computational budgets [25].

2. Future Directions and Open Challenges

Generalization and Robustness: For safety-critical applications, strengthening the model's resistance to adversarial attacks and domain transformations (such as varied weather conditions) is essential. Learning with Less Supervision: Using self-supervised, semi-supervised, and weakly-supervised learning techniques to lessen reliance on expensive bounding-box annotations [26]. Video Object Detection and Tracking: Extending detection to video streams in order to interact with multi-object tracking and manage temporal issues like occlusion. 3D Object Detection: Predicting 3D bounding boxes from different sensor inputs (cameras, LiDAR) is a critical area for robotics and autonomy [27].

VII. CONCLUSION

In conclusion, these foundational and modern models define the basic limits and ongoing evolution of the object detection design space. The once-distinct two-stage and one-stage paradigms have progressively incorporated advanced features like multi-scale feature pyramids and attention mechanisms, a trend that culminated in the paradigm-shifting introduction of the Vision Transformer (ViT). The subsequent convergence of these ideas, exemplified by the leap from the refined CNN architecture of YOLOv8 to the hybrid, transformer-infused design of YOLOv11, laid the groundwork for the current wave of innovation that continues to challenge the fundamental speed-accuracy trade-off.

REFERENCES

- [1] V. Nain, H. S. Shyam, N. Kumar, P. Tripathi, and M. Rai, "A study on object detection using artificial intelligence and image processing-based methods," in *Mathematical Models Using Artificial Intelligence for Surveillance Systems*. Boca Raton, FL, USA: CRC Press, 2024, pp. 121-148.
- [2] M. Hirz and B. Walzel, "Sensor and object recognition technologies for self-driving cars," *Computer-Aided Design and Applications*, vol. 15, no. 4, pp. 501-508, 2018.
- [3] R. Yang and Y. Yu, "Artificial convolutional neural network in object detection and semantic segmentation for medical imaging analysis," *Frontiers in Oncology*, vol. 11, p. 638182, 2021.
- [4] S. Jha et al., "Real time object detection and tracking system for video surveillance system," *Multimedia Tools and Applications*, vol. 80, no. 3, pp. 3981-3996, 2021.
- [5] Z. Chen, K. Chen and J. Chen, "Vehicle and pedestrian detection using support vector machine and histogram of oriented gradients features," *2013 International Conference on Computer Sciences and Applications*, Wuhan, China, 2013, pp. 309-311, IEEE.
- [6] H. Sharma et al., "Analyzing impact of image scaling algorithms on viola-jones face detection framework," in *Proc. 2015 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, Kochi, India, 2015, pp. 1642-1647.
- [7] L. Du, R. Zhang and X. Wang, "Overview of two-stage object detection algorithms," in *Journal of Physics: Conference Series*, vol. 1544, no. 1, IOP Publishing, 2020, p. 012033.
- [8] L. Yang et al., "PDNet: Toward better one-stage object detection with prediction decoupling," *IEEE Transactions on Image Processing*, vol. 31, pp. 5121-5133, 2022, doi: 10.1109/TIP.2022.3192720.
- [9] Z. Zhao, P. Zheng, S. Xu and X. Wu, "Object detection with deep learning: A review," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, no. 11, pp. 3212-3232, 2019.
- [10] S. Ma and Y. Xu, "Mpdious: a loss for efficient and accurate bounding box regression," *arXiv preprint arXiv:2307.07662*, 2023.
- [11] R. Padilla, S. L. Netto and E. A. B. Da Silva, "A survey on performance metrics for object-detection algorithms," *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*, Niterói, Brazil, 2020, pp. 237-242, IEEE.
- [12] J. Lee et al., "Re-VoxelDet: Rethinking neck and head architectures for high-performance voxel-based 3d detection," in *Proc. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024, pp. 790-799.
- [13] J. Hosang, R. Benenson and B. Schiele, "Learning non-maximum suppression," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 4509-4517.
- [14] R. Girshick, J. Donahue, T. Darrell and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Columbus, OH, USA, 2014, pp. 580-587.
- [15] R. Girshick, "Fast R-CNN," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1440-1448.
- [16] S. Ren, K. He, R. Girshick and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [17] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779-788.
- [18] J. Terven, D. Córdova-Esparza, and J. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Mach. Learn. Knowl. Extr.*, vol. 5, no. 4, pp. 1680-1716, 2023.
- [19] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," *arXiv preprint arXiv:2410.17725*, 2024.
- [20] W. Liu et al., "SSD: Single shot multibox detector," in *Proc. European Conference on Computer Vision (ECCV)*, Amsterdam, The Netherlands, 2016, pp. 21-37, Springer.
- [21] Z. Tian, C. Shen, H. Chen and T. He, "FCOS: A simple and strong anchor-free object detector," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 4, pp. 1922-1933, 2022.
- [22] T. Wang et al., "FCOS3D: Fully convolutional one-stage monocular 3d object detection," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 913-922.
- [23] T. Shehzaadi et al., "Object detection with transformers: A review," *Sensors*, vol. 25, no. 19, p. 6025, 2025.
- [24] X. Zhu et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," *arXiv preprint arXiv:2010.04159*, 2020.
- [25] R. Zhang et al., "Optimization Methods, Challenges, and Opportunities for Edge Inference: A Comprehensive Survey," *Electronics*, vol. 14, no. 7, p. 1345, 2025.
- [26] S. Mohseni et al., "Taxonomy of Machine Learning Safety: A Survey and Primer," *ACM Comput. Surv.*, vol. 55, no. 8, pp. 1-38, 2022.
- [27] S. Y. Alaba and J. E. Ball, "A Survey on Deep-Learning-Based LiDAR 3D Object Detection for Autonomous Driving," *Sensors*, vol. 22, no. 24, p. 9577, 2022.
- [28] J. Yang et al., "Focal Self-Attention for Local-Global Interactions in Vision Transformers," *arXiv preprint arXiv:2107.00641*, 2021.